

Symbolic Manipulation in Projective Geometry through the Cooperation of Dynamic Geometry Systems and Computer Algebra Systems*

Eugenio Roanes-Lozano* and Eugenio Roanes-Macías*

* Universidad Complutense de Madrid
Departamento de Álgebra, Facultad de Educación
c/ Rector Royo Villanova s/n; E-28040-Madrid (España)
`{eroanes,roanes}@mat.ucm.es`

Extended abstract

Introduction

Dynamic Geometry Systems (DGS) were developed with the intention of allowing the user to “explore” (plane) Euclidean Geometry the same way he would proceed with (graduated) rule and compass. The adjective *dynamic* comes from the fact that, once the sketch is finished, the first objects drawn (points) can be dragged and dropped with the mouse, consequently changing the whole sketch.

Several DGS exist nowadays: *The Geometer’s Sketchpad (GSP)*, *Cabri Geometry II*, *Cinderella*, *Euklid*, *Dr. Geo*, *WinGeom*, *The Geometric Supposer*...

Cinderella offers the possibility to deal with non-Euclidean Geometries. Moreover, it internally considers the affine plane embedded in a projective plane ([1], p. 48). Internal calculations, like the system solving necessary to find intersections, are computed in $\mathcal{C}$ ([1], p. 92). Both things with the purpose to avoid the drawing from vanishing in exceptional cases.

Unfortunately, Computer Algebra Systems (CAS) and DGS have evolved independently. Some CAS, like Maple, include powerful packages devoted to Euclidean Geometry, but no one includes dynamic capabilities.

On the other hand, DGS cannot deal with non-assigned variables. This fact prevents the use of DGS in any process involving non-assigned variables, like mechanical theorem proving in Geometry (where it is standard and necessary to consider parameters as certain coordinates of the initial points).

This need for cooperation was already underlined in [2]. The case of some calculators, like the *TI-92*, where both technologies are simultaneously available but are isolated, is really surprising.

*Partially supported by research project TIC-2000-1368-C03-03, Ministry of Science and Technology (MCYT), Spain.

Other authors have solved these lacks in different ways, but all the approaches we know are based in implementing new software (whole DGS and/or CAS!):

- *The Algebraic Geometer* [3], announced for some time, but not available
- *GDI* (unifying the former *Lugares*, *Discovery* and *Rex* [4], are new Spanish DGS implemented in *Visual-Prolog* by J.L. Valcarce and F. Botana, specialized in determining geometric loci and automatic theorem proving and discovery, by calling CoCoA or Mathematica for performing algebraic computations
- *Geometry Expert* [5], developed in China by X.S. Gao, is a new DGS with algebraic capabilities and has a similar orientation. The user can choose among different methods for automatic proving. It has been abandoned in favor of a projective version now under development.
- *MathXP* [6] is another Chinese package, also incorporating a DGS and symbolic manipulation. This one doesn't use the standard methods for mechanical theorem proving in Geometry (Gröbner Bases, Wu's...), but an inference engine to perform inference and a CAS to solve systems.

Moreover, they are oriented as software packages that exceed in a certain task but are not flexible and do not allow the user to take control of the processes.

Our strategy is completely different: software reuse. Another difference is that we do not try to produce a system that exceeds in a certain task (e.g. mechanical theorem proving in Geometry) but a kind of dynamic Graphic User Interface (GUI) with diverse applications.

We have developed a bridge between existing DGS and CAS: *GSP v.3* & *v.4* and *Maple 8* and *Derive 5*.

The key idea is simple:

- i) to use the almost natural language description of *GSP v.3*'s scripts or the .HTM output for *JavaSketch* of *GSP v.4*'s sketches
- ii) to translate them to the CAS *Maple*'s or *Derive*'s syntax using an external translator developed *ad hoc*
- iii) to interpret the translated code thanks to new geometrical packages for *Maple* and *Derive*, denoted *paramGeo*, that can handle parameters and exactly translate the possibilities of *GSP*'s menus.

For example, the construction of Desargues' Theorem (Fig. 1):

$$M, N, Q \text{ are collinear} \Leftrightarrow AU, BV, CW \text{ share a common point}$$

produces the following *script* in *GSP v.3*:

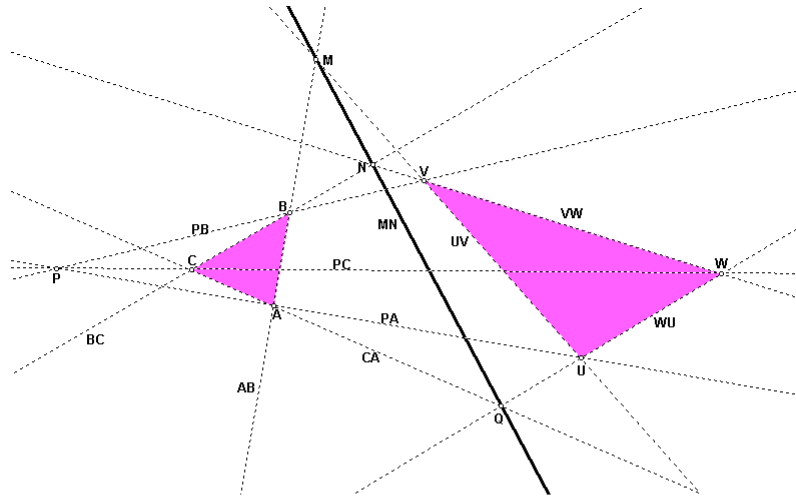


Figure 1: Desargues' Theorem

desargues.gss

Given:

Point A
Point B
Point C
Point U
Point V
Point W

Steps:

1. Let AB = Line between Point A and Point B.
2. Let BC = Line between Point B and Point C.
3. Let CA = Line between Point C and Point A.
4. Let UV = Line between Point U and Point V.
5. Let VW = Line between Point V and Point W.
6. Let WU = Line between Point W and Point U.
7. Let AU = Line between Point A and Point U.
8. Let BV = Line between Point B and Point V.
9. Let CW = Line between Point C and Point W.
10. Let M = Intersection of Line AB and Line UV.
11. Let N = Intersection of Line BC and Line VW.
12. Let Q = Intersection of Line CA and Line WU.
13. Let P = Intersection of Line AU and Line BV.
14. Let MN = Line between Point M and Point N.

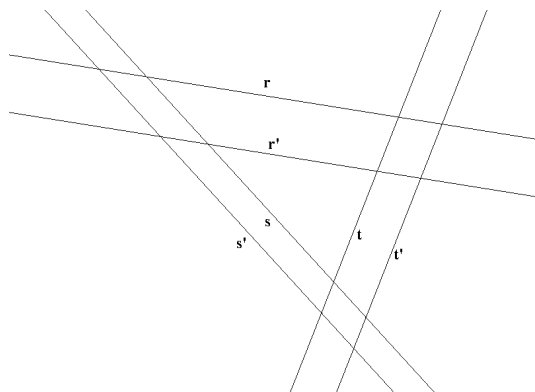


Figure 2: Treatment I

A first version of our package for *GSP 3* and *Maple 7* was presented at ACA'2001 and a preprint (in Spanish) is available from [7] (it used a translator written in *Maple*). A much improved version for *GSP v.3 & v.4*, using an external translator written in *Visual-Basic*, was presented at ACA'2002. The *Derive* version was presented at VisiT-me'2002 conference. But we have been thinking about these problems for quite a long time now. The skeleton was presented in 2000 in the previous International Derive Conference. A discussion about the different solutions to the problem was presented as a plenary talk at ICTMT5 [8] in 2001. The final package for Plane Affine Geometry (updated for *Maple 8*) will appear in "Mathematical and Computer Modeling" journal [9].

The developers of *GSP* are considering the possibility to include this package in their DGS.

The projective case

There are a few DGS that can treat the projective case. One is *pdb*, presented in the excellent thesis [10]. It works internally in $\mathcal{C}$, but the GUI corresponds to a projective real plane. Unfortunately it hasn't been spread.

Other systems with projective capabilities are the constraint-based prototypes *GéoSpécif* [11] and *UniGéom*. Another one is PyGeo [12], developed in *Python* language, which GUI is only used for the dynamic possibilities (its constructions have to be introduced in a script-like description). Finally, *Juno-2* [13] is another constraint-based DGS with projective capabilities. It has a double editor, representing both the figure and its internal representation, and both can be altered. None of them have algebraic capabilities.

As said above, X.S. Gao has stopped the development of *Geometry Expert* and is now preparing a projective DGS (with the same orientation), that uses Plücker coordinates.

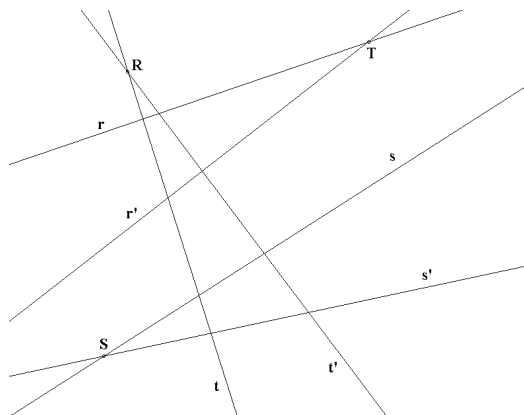


Figure 3: Treatment II

Our strategy of software reuse is also applicable in the projective case. We use *GSP v.3* or *v.4* as a GUI for the CAS where the points of an affine plane, embedded in a projective plane are represented. The computations performed with the algebraic expressions returned by the DGS are performed in the projective plane. Even the same translator is used. The symbolic packages in the CAS have, of course, been rewritten.

Projective Geometry theorems that are also true in Affine Geometry (like Desargues, Pascal...) can be obviously introduced, as their configuration can be represented in the affine plane.

The dynamic exploration has a clear restriction: for instance it is not possible to check that a certain theorem, where two lines are intersected at a certain step, is also true when these two lines are parallel.

Observe that if the configuration included imaginary points, the reference system could be changed in order the initial points not to lie on the infinite line. So the DGS could still be used as a GUI for the CAS, what would save a lot of time.

Let us detail an specially problematic example: let r, r', s, s', t, t' be lines in the projective plane. If $r \parallel r', s \parallel s'$ and $t \parallel t'$, then $R = r \cap r', S = s \cap s'$ and $T = t \cap t'$ are collinear (they all lie on the infinite line).

One solution would be to draw three pairs of parallel lines (Fig. 2). As the DGS doesn't allow to intersect them (as it works in the affine plane), we could ask the CAS to intersect them in the projective plane and to check algebraically that the solution points are aligned. But if the construction went on behind this step, it is clear that the process in the DGS used as a GUI for the CAS would be interrupted at this step, what is not desirable.

A second solution, more general and clearly preferable if the construction went on, would be not to include the parallelism condition in the configuration (Fig. 3), with the purpose of having explicitly the intersection points in the

DGS (the parallelism conditions would be included manually in the CAS later). The inconvenience of this approach is that the image in the sketch is no longer intuitive.

Apart from the obvious fact that we can treat Projective Geometry problems, there is another advantage in this method. When a configuration is expressed algebraically, not only polynomials but also rational expressions can be obtained. The coordinates of the free points (variables) appear in the numerators and the coordinates of the derived points (parameters) can appear both in numerators and denominators. This denominators are related to the existence of “degenerated cases”. Dealing with projective coordinates suppresses the degeneracies due to parallelism, and therefore denominators disappear, or, at least, become simpler.

Our approach has several possibilities:

- i) to use the package directly from a *Maple* or *Derive* session in order to perform algebraic calculations in Projective Geometry
- ii) to sketch geometric constructions with the mouse in the DGS and to obtain the corresponding algebraic expressions (in the projective plane)
- iii) to determine the equations of a geometric locus of the projective plane (in a constructive way)
- iv) to perform mechanical theorem proving in Projective Geometry
- v) to perform automatic discovery in Projective Geometry.

For instance, the mechanical theorem proving of the direct implication ($\Rightarrow$) of Desargues Theorem (Fig. 1) would consist in translating the script into Maple code with the translator, to establish the hypothesis and thesis polynomials:

```
> cond_1:=isPlaced(P,CW);
> cond_2:=isPlaced(Q,MN);
```

and to check that the thesis is a multiple of the hypothesis (because this would imply that the thesis vanishes whenever the hypothesis vanishes):

```
> divide(cond_2,cond_1);
      true
```

The inverse implication ($\Leftarrow$) can be proven similarly.

Let us observe that this is the simplest case, as there are no point membership conditions. Intuitively, such conditions are usually hypotheses. That the thesis vanishes when the hypotheses vanish, can be proven using Ideals Theory (applying Gröbner Bases or Wu’s methods): it has to be checked if the thesis is an algebraic linear combination of the hypotheses. This is the case for instance in Pascal’s Theorem (Fig. 4).

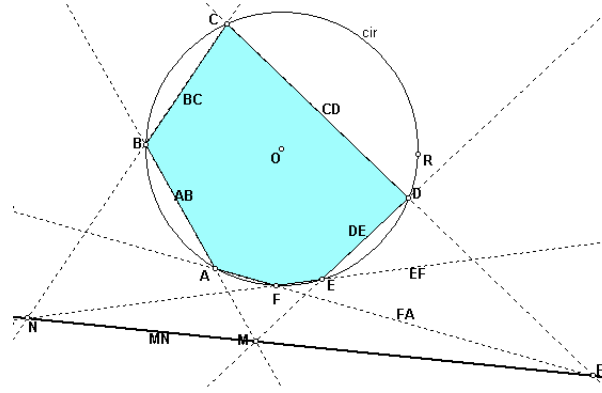


Figure 4: Pascal's Theorem

Conclusions

We think this tool can be very useful and convenient when dealing with manipulations in the projective plane. Although projective dynamic capabilities are not fully available, it still has a didactic application, with the usual advantages of educational simulations: to allow the student to save time, to explore many more situations and to concentrate in the ideas (leaving the mechanical computations to the computer).

Key words: Projective Geometry, Symbolic Computation, Dynamic Geometry, Mechanical Theorem Proving.

References

- [1] U. Kortenkamp: *Foundations of Dynamic Geometry* (PhD. Thesis), Swiss Fed. Inst. Tech. Zurich, 1999.
- [2] T. Recio: *Cálculo Simbólico y Geométrico*, Ed. Síntesis, 1998.
- [3] URL: <http://www.saltire.com/paraprts.html>
- [4] J.L. Valcarce, F. Botana: *Lugares. Manual de Referencia*. Universidad de Vigo, 2001.
- [5] URL: <http://www.mmrc.iss.ac.cn/~xgao/gex.html>
- [6] H. Fu, Z. Zeng: *A New Dynamic Geometry Software with a Prover and a Solver*. En *Proceedings of VisiT-me'2002 – Vienna Intl. Symposium on Integrating Technology into Mathematics Teaching* (CD-ROM). BK-Teachware, 2002.

- [7] URL: <http://www.us.es/da/prepubli/nsprep14.pdf>
- [8] E. Roanes-L.: *Boosting the Geometrical Possibilities of Dynamic Geometry Systems and Computer Algebra Systems through Cooperation*. En: M. Borovcnick, H. Kautschitsch (eds.): *Technology in Mathematics Teaching. Procs. of ICTMT-5* (335-348). Schrifrenreihe Didaktik der Mathematik v.25, öbv&hpt, 2002.
- [9] E. Roanes-L., E. Roanes-M., M. Villar: A Bridge Between Dynamic Geometry and Computer Algebra. *Mathematical and Computer Modelling*. Accepted for publication (to appear during 2003).
- [10] H. Winroth: *Dynamic Projective Geometry* (PhD. Thesis), Stockholms Universitet, 1999.
- [11] R.J. Allen, L. Trilling: *Dynamic Geometry and Declarative Geometric Programming*. In: J. King , D. Schattschneider (eds.): *Geometry Turned On* (193-197). The Math. Assoc. of America, 1997.
- [12] URL:<http://home/netcom.com/~asj/>
- [13] URL:<http://research.compaq.com/SRC/juno-2/>